МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

Інститут КНІТ
Кафедра ПЗ

ЗВІТ

До лабораторної роботи № 4
З дисципліни: *“Кросплатформне програмування”*
На тему: *“Реалізація автентифікації та інтеграція з Firebase”*

Лектор:
ст. викл. каф. ПЗ
Угриновський Б. В.

Виконав:
студент гр. ПЗ-34
Данюк Ю. І.

Прийняв:
асистент каф. ПЗ
Гавдульський Р. І.

Львів – 2026

Тема роботи: Реалізація автентифікації та інтеграція з Firebase.

Мета роботи: Навчитися інтегрувати Firebase у Flutter-застосунок та реалізовувати автентифікацію користувачів.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Залежності Flutter-проєкту описуються у pubspec.yaml. Пакети для роботи застосунку додають у dependencies, а засоби розробки та тестування — у dev_dependencies. Обмеження версій визначають допустимий діапазон, а pubspec.lock фіксує конкретні версії прямих і транзитивних залежностей. Це дає змогу відтворювати однакове середовище під час розробки.

Плагіни поєднують Dart API з реалізаціями для цільових платформ. Для взаємодії Dart із нативним кодом використовують платформні канали. MethodChannel передає назву операції й аргументи нативному обробнику та асинхронно повертає результат. Перед використанням плагіна потрібно перевірити підтримку платформи і виконати її налаштування.

Firebase надає окремі сервіси для автентифікації, аналітики та діагностики. Пакет firebase_core забезпечує ініціалізацію застосунку, а згенерований firebase_options.dart містить параметри його зв’язку з Firebase-проєктом. Звертатися до сервісів слід після завершення Firebase.initializeApp().

Firebase Authentication створює облікові записи та перевіряє облікові дані. Для email і пароля використовують createUserWithEmailAndPassword() та signInWithEmailAndPassword(). Стан користувача можна відстежувати через userChanges(). Поточний користувач має uid, email, displayName і photoURL. Оновлення імені або посилання на аватар не потребує створення окремої колекції у базі даних.

Сеанс підтримується Firebase SDK за допомогою токенів. Після повторного запуску застосунок очікує початковий стан автентифікації, а потім показує екран входу або головний екран. Вихід через signOut() завершує локальний сеанс. Пароль не слід зберігати у власній моделі користувача або записувати в журнали.

Зміна пароля в активному сеансі відрізняється від відновлення забутого пароля. У першому випадку користувач підтверджує особу через reauthenticateWithCredential(), після чого виконується updatePassword(). У другому випадку sendPasswordResetEmail() надсилає лист із посиланням на встановлення нового пароля. Зміна photoURL зберігає лише HTTPS-посилання, а не завантажує файл у сховище.

Firebase Analytics реєструє події використання застосунку. Частину подій SDK збирає автоматично, а прикладні дії й переходи між Flutter-екранами потрібно передавати з коду. Для цього доступні logLogin(), logSignUp(), logScreenView() і logEvent(). DebugView допомагає перевірити доставку подій під час розробки.

Firebase Crashlytics об’єднує повідомлення про помилки у звіти зі стеком викликів, версією застосунку та інформацією про середовище. Помилки Flutter перехоплюють через FlutterError.onError, а необроблені помилки кореневого ізоляту — через PlatformDispatcher.instance.onError. recordError() створює звіт; log() лише доповнює діагностичний журнал. Позначка fatal визначає класифікацію звіту й сама по собі не завершує процес.

Віджет Form об’єднує поля TextFormField. Ключ GlobalKey<FormState> зберігають у State, щоб він не змінювався під час кожної побудови. Метод validate() викликає валідатори: null означає коректне значення, а рядок — повідомлення про помилку. Локальна перевірка обов’язковості й формату не замінює серверну перевірку пароля, існування акаунта та мережевих помилок.

Для відокремлення інтерфейсу від Firebase застосовують репозиторій. Екран викликає метод AuthRepository, показує завантаження й обробляє результат. Поки запит виконується, повторне надсилання блокують; після помилки зберігають введені дані та дозволяють повторну спробу. Такий поділ дає змогу тестувати інтерфейс із підставним репозиторієм без реальних запитів.

ЗАВДАННЯ

Варіант № 4 — «Помічник із догляду за рослинами». Застосунок Sprout розвиває інтерфейс із лабораторної роботи № 3 та функціональні вимоги лабораторної роботи № 2.

1. Виконати інтеграцію Firebase Analytics. У звіті відобразити сторінку аналітики Firebase для свого проєкту. Якщо цільова платформа не підтримується, використати Amplitude Analytics або інший аналог.

2. Виконати інтеграцію Firebase Crashlytics. Згенерувати одну помилку, яка відобразиться у Firebase Crashlytics. У звіті відобразити сторінку звіту про помилки свого проєкту. За відсутності підтримки платформи допускається Sentry.io або інший аналог.

3. Налаштувати Firebase Auth відповідно до вимог лабораторної роботи № 2: а) обов’язково реалізувати основний метод email/password для реєстрації і входу; б) за потреби об’єднати кілька способів входу в один акаунт, якщо передбачено соціальну автентифікацію. У звіті відобразити сторінку з налаштованими способами входу.

4. Реалізувати бізнес-логіку взаємодії з Firebase Auth: а) реєстрацію, вхід і відновлення пароля через електронну пошту; б) керування профілем зі зміною пароля. Функції, що потребують Cloud Firestore або push-сповіщень, допускається залишити демонстраційними. Зміну фото реалізувати вибором готових HTTPS-зображень або введенням власного HTTPS-посилання з попереднім переглядом; завантаження файлів у сховище відкладено до лабораторної роботи № 6.

Забезпечити коректну роботу з правильними та неправильними даними: повідомлення про невдалий вхід і реєстрацію; стан завантаження та блокування повторного надсилання; успішний вхід, вихід і відновлення сеансу; помилки зміни пароля, зокрема потребу повторного входу. Винести взаємодію з Firebase в окремий клас AuthRepository. У звіті навести код репозиторію та один приклад виклику його методу з відображенням даних.

5. Додати валідацію полів через Form на екранах реєстрації та входу: обов’язкове ім’я, правильний формат email та інші відповідні перевірки. У звіті відобразити код форми реєстрації. Оновити також інші попередньо створені екрани, які потребують перевірки введених даних.

6. Відобразити у звіті зміст файлу main.dart після всіх змін.

7. Відобразити у звіті структуру проєкту після всіх змін.

ХІД РОБОТИ

1 Інтеграція Firebase Analytics

Для Sprout створено Firebase-проєкт sprout-a6523 та зареєстровано iOS-застосунок із Bundle ID com.example.sprout. У цій роботі Firebase використано замість попередньо запропонованого у лабораторній № 2 Supabase. Цільова платформа застосунку — iOS 15.0 і новіші версії.

FlutterFire CLI сформував firebase_options.dart і конфігурацію iOS. У pubspec.yaml підключено firebase_core 5.0.0, firebase_auth 7.0.0, firebase_analytics 13.0.0 та firebase_crashlytics 6.0.0; ці версії зафіксовано у pubspec.lock. Ініціалізація завершується до створення репозиторію та використання сервісів.

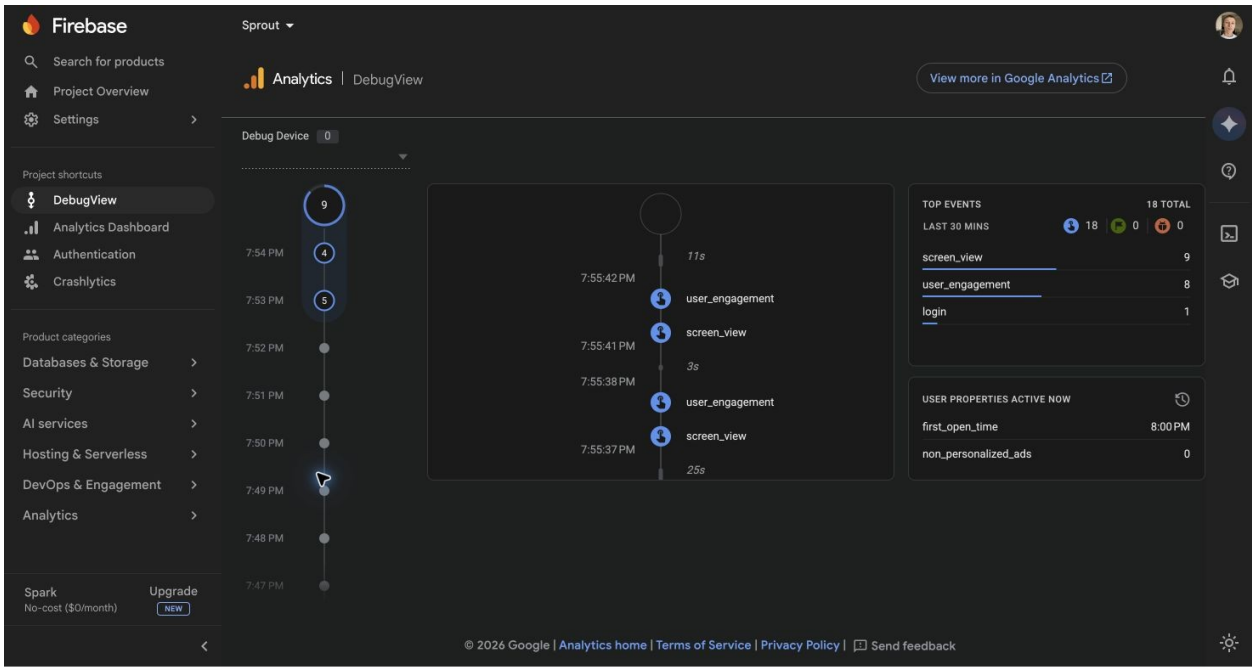
AnalyticsService надсилає події login і sign_up після успішних відповідних операцій. Метод screen() реєструє screen_view. RouteAware відстежує відкриття та повторний показ маршруту, а перемикання Home/Profile обробляється окремо. Повторна побудова віджета сама по собі не створює нової події. Email і паролі у параметри подій не передаються.

Файл: lib/core/services/analytics_service.dart

```
import 'package:firebase_analytics/firebase_analytics.dart';

class AnalyticsService {
  const AnalyticsService([this.client]);
  final FirebaseAnalytics? client;
  Future<void> screen(String name) => _safely(() async {
    await client?.logScreenView(screenName: name);
  });
  Future<void> login() => _safely(() async {
    await client?.logLogin(loginMethod: 'password');
  });
  Future<void> signup() => _safely(() async {
    await client?.logSignUp(signUpMethod: 'password');
  });
  Future<void> _safely(Future<void> Function() action) async {
    try {
      await action();
    } catch (_) {
      /* Telemetry must not block authentication. */
    }
  }
}
```

Під час перевірки DebugView отримав події screen_view, login та user_engagement. На рис. 1 наведено сторінку Analytics проєкту Sprout із фактично доставленими подіями.



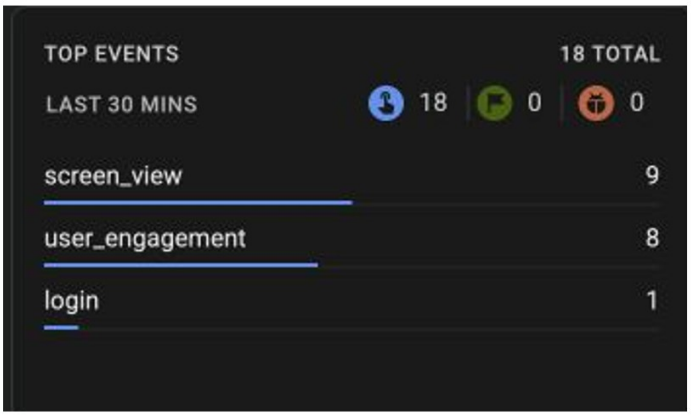


Рис. 1. Firebase Analytics DebugView та збільшений фрагмент подій

2 Інтеграція Firebase Crashlytics

CrashReportingService встановлює обробники помилок Flutter і кореневого ізоляту. Для перевірки інтеграції створено контрольну нефатальну помилку з повідомленням «Sprout laboratory 4 controlled non-fatal test». Виклик доступний лише у debug-збірці за параметром CRASHLYTICS_TEST; звичайний запуск його не виконує.

Файл: lib/core/services/crash_reporting_service.dart

```
import 'package:firebase_crashlytics/firebase_crashlytics.dart';
import 'package:flutter/foundation.dart';

abstract final class CrashReportingService {
  static void install() {
    FlutterError.onError = (details) {
      FlutterError.presentError(details);

FirebaseCrashlytics.instance.recordFlutterFatalError(details);
    };
    PlatformDispatcher.instance.onError = (error, stack) {
      FirebaseCrashlytics.instance.recordError(error, stack,
fatal: true);
      return true;
    };
  }

  static Future<void> sendTestError() async {
    if (!kDebugMode) return;
    await FirebaseCrashlytics.instance.recordError(
      StateError('Sprout laboratory 4 controlled non-fatal test'),
      StackTrace.current,
      reason: 'Intentional laboratory verification',
      fatal: false,
    );
    await FirebaseCrashlytics.instance.sendUnsentReports();
  }
}
```

Після надсилання звіту та завантаження символів налагодження у Firebase відобразився читабельний стек: crash_reporting_service.dart:20 та main.dart:38. На рис. 2 видно контрольну нефатальну помилку; застосунок після її реєстрації продовжив роботу.

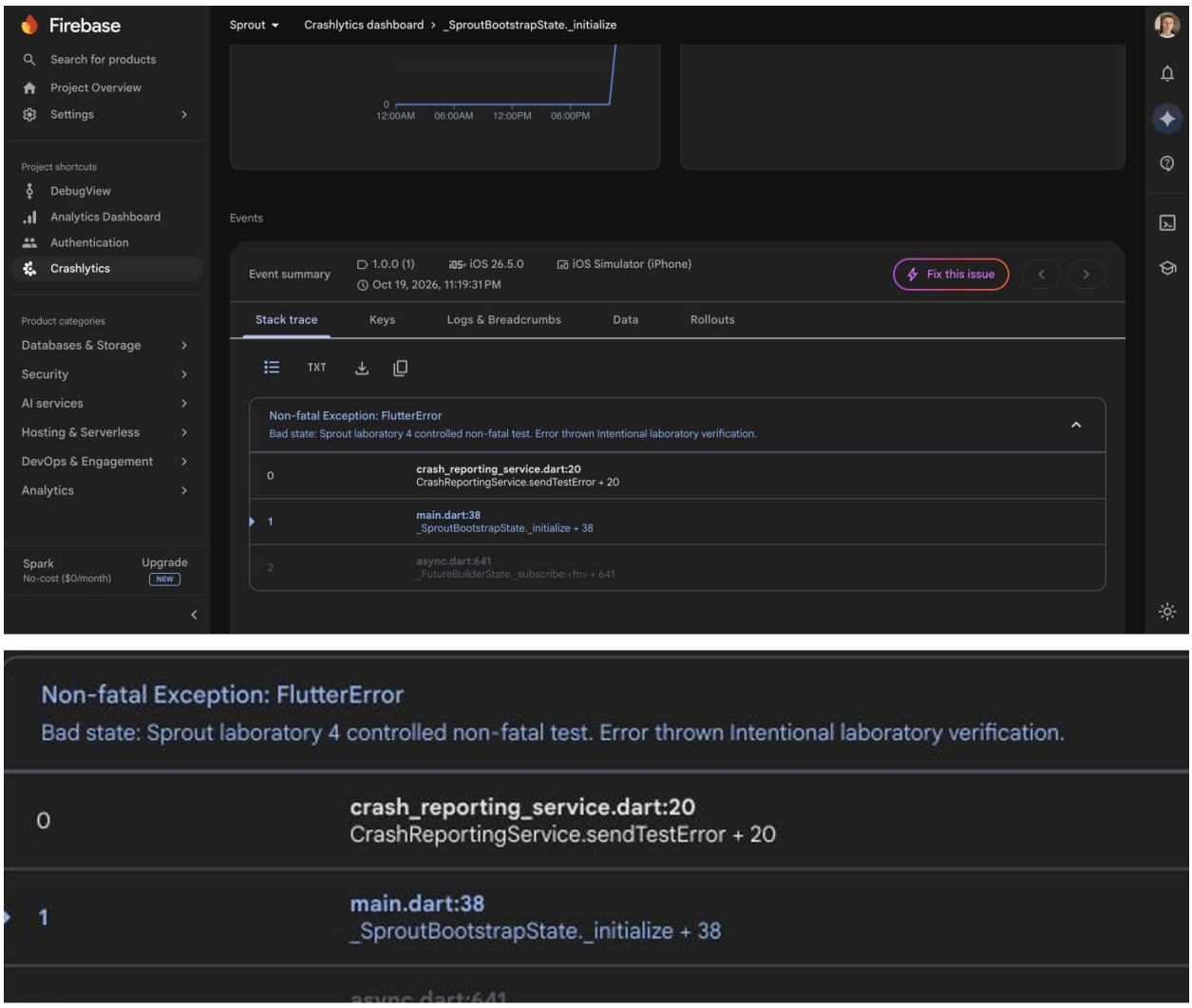


Рис. 2. Firebase Crashlytics та збільшений фрагмент контрольної помилки

3 Налаштування Firebase Authentication

У Firebase Authentication увімкнено провайдер Email/Password, що показано на рис. 3. Соціальні способи входу не були передбачені вимогами Sprout, тому зв’язування Google, Apple або Facebook з акаунтом у цій роботі не потрібне.

Для нового пароля погоджено мінімум 8 символів і щонайменше одну велику латинську літеру A–Z. Правило застосовано у формах реєстрації та зміни пароля й узгоджено з політикою Firebase. На вході перевіряється наявність пароля; його правильність визначає Firebase.

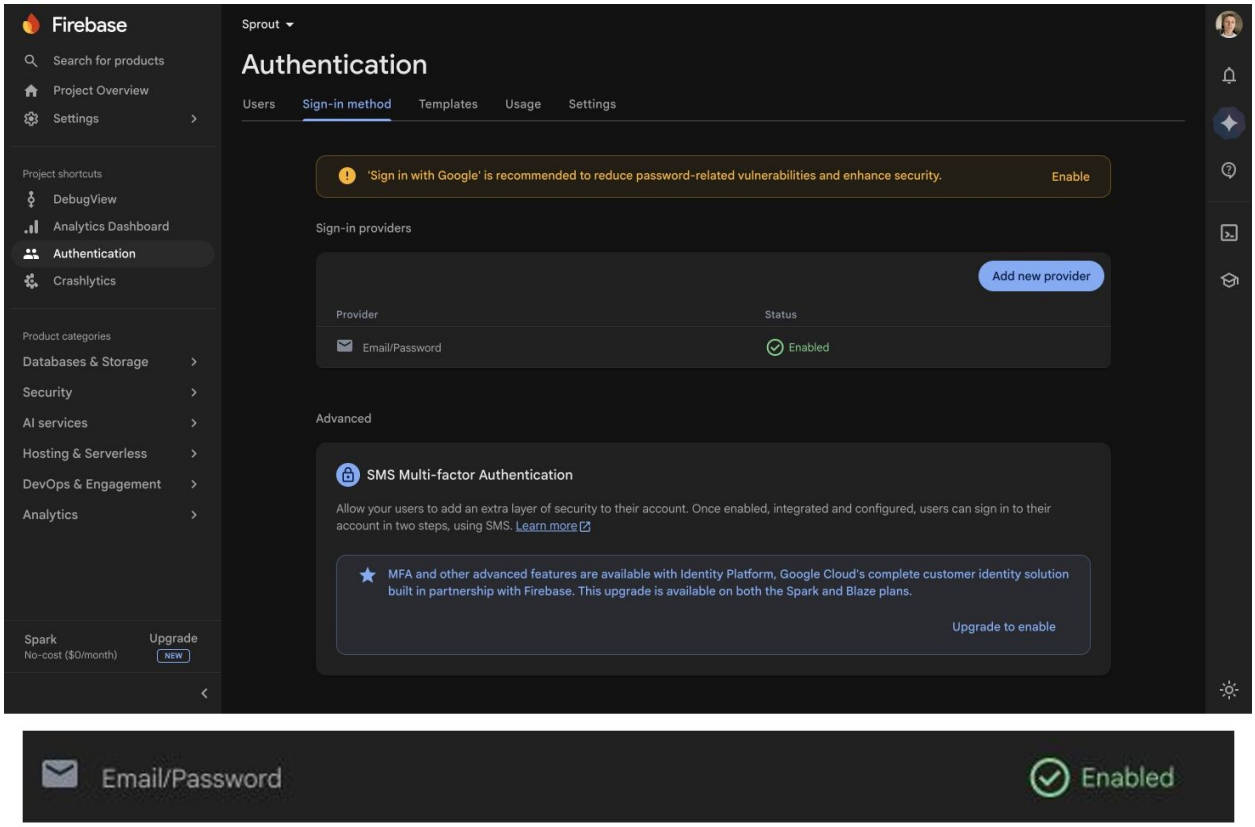


Рис. 3. Email/Password для Sprout та збільшений фрагмент статусу Enabled

Успішна реєстрація створює справжній акаунт Firebase та зберігає ім’я. Вхід відкриває Home, вихід очищає поточний стек екранів, а Firebase SDK відновлює сеанс після повторного запуску. AuthGate очікує початковий стан та визначає потрібний екран. Пароль не входить у модель AppUser.

4 Автентифікація та керування профілем

Взаємодію із сервером винесено в AuthRepository та його реалізацію FirebaseAuthRepository. Репозиторій перетворює Firebase User у власну модель AppUser, повідомляє слухачів про зміни та надає зрозумілі повідомлення замість технічного тексту винятків.

Реєстрація враховує частковий успіх: якщо акаунт створено, але ім’я не збережено, повторна спроба завершує оновлення профілю. Зміна пароля спочатку повторно підтверджує поточні облікові дані. Оновлення імені одразу публікується для Home та Profile, зокрема якщо наступна операція з фото завершилася помилкою.

Файл: lib/features/auth/data/auth_repository.dart

```
import 'dart:async';

import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/foundation.dart';

import '../../session/app_user.dart';

class AuthFailure implements Exception {
  const AuthFailure(this.message);
  final String message;
  @override
  String toString() => message;
}

abstract class AuthRepository extends ChangeNotifier {
  AppUser? get currentUser;
  bool get initializing;
  Future<void> signUp({
    required String name,
    required String email,
    required String password,
  });
  Future<void> signIn({required String email, required String
password});
  Future<void> signOut();
  Future<void> sendPasswordReset({required String email});
  Future<void> updateProfile({required String name, required
String? photoUrl});
  Future<void> changePassword({
    required String currentPassword,
    required String newPassword,
  });
}

class FirebaseAuthRepository extends AuthRepository {
```

```
  FirebaseAuthRepository({FirebaseAuth? auth})
    : _auth = auth ?? FirebaseAuth.instance {
    _subscription = _auth.userChanges().listen(
      (user) {
        if (_registering) return;
        _publish(user);
      },
      onError: (Object _) {
        _initializing = false;
        notifyListeners();
      },
    );
  }
  final FirebaseAuth _auth;
  late final StreamSubscription<User?> _subscription;
  AppUser? _user;
  bool _initializing = true;
  bool _registering = false;
  String? _incompleteSignupUid;
  @override
  AppUser? get currentUser => _user;
  @override
  bool get initializing => _initializing;

  void _publish(User? user) {
    _initializing = false;
    _user = user == null
        ? null
        : AppUser(
            uid: user.uid,
            name: user.displayName ?? '',
            email: user.email ?? '',
            photoUrl: user.photoURL,
          );
    notifyListeners();
  }

  Future<void> _request(Future<void> Function() action) async {
    try {
      await action();
    } on FirebaseAuthException catch (error) {
      throw AuthFailure(_message(error.code));
    } on AuthFailure {
      rethrow;
    } catch (_) {
      throw const AuthFailure('Something went wrong. Please try
again.');
    }
  }

  @override
  Future<void> signUp({
    required String name,
```

```
    required String email,
    required String password,
  }) => _request(() async {
    _registering = true;
    try {
      User? user = _auth.currentUser;
      if (_incompleteSignupUid == null ||
          user?.uid != _incompleteSignupUid ||
          user?.email != email.trim()) {
        user = (await _auth.createUserWithEmailAndPassword(
          email: email.trim(),
          password: password,
        )).user!;
        _incompleteSignupUid = user.uid;
      }
      try {
        await user!.updateDisplayName(name.trim());
      } on FirebaseAuthException {
        throw const AuthFailure(
          'Your account was created, but your name could not be
saved. Tap Sign Up again to finish.',
        );
      }
      _incompleteSignupUid = null;
      _publish(_auth.currentUser);
    } finally {
      _registering = false;
    }
  });

  @override
  Future<void> signIn({required String email, required String
password}) =>
      _request(() async {
        await _auth.signInWithEmailAndPassword(
          email: email.trim(),
          password: password,
        );
        _publish(_auth.currentUser);
      });
  @override
  Future<void> signOut() => _request(() async {
    await _auth.signOut();
    _incompleteSignupUid = null;
    _publish(null);
  });
  @override
  Future<void> sendPasswordReset({required String email}) =>
_request(() async {
    try {
      await _auth.sendPasswordResetEmail(email: email.trim());
    } on FirebaseAuthException catch (e) {
      if (e.code != 'user-not-found') rethrow;
```

```
    }
  });
  @override
  Future<void> updateProfile({
    required String name,
    required String? photoUrl,
  }) => _request(() async {
    final user = _auth.currentUser;
    if (user == null) throw const AuthFailure('Please log in
again.');
    try {
      await user.updateDisplayName(name.trim());
      await user.updatePhotoURL(photoUrl);
    } finally {
      _publish(_auth.currentUser);
    }
  });
  @override
  Future<void> changePassword({
    required String currentPassword,
    required String newPassword,
  }) => _request(() async {
    final user = _auth.currentUser;
    if (user == null || user.email == null) {
      throw const AuthFailure('Please log in again.');
    }
    try {
      await user.reauthenticateWithCredential(
        EmailAuthProvider.credential(
          email: user.email!,
          password: currentPassword,
        ),
      );
    } on FirebaseAuthException catch (error) {
      if (error.code == 'invalid-credential' ||
          error.code == 'wrong-password') {
        throw const AuthFailure('Your current password is
incorrect.');
      }
      rethrow;
    }
    await user.updatePassword(newPassword);
  });

  static String _message(String code) => switch (code) {
    'invalid-credential' ||
    'wrong-password' ||
    'user-not-found' ||
    'invalid-login-credentials' =>
      'Unable to log in. Check your email and password.',
    'email-already-in-use' =>
      'This email is already registered. Log in or reset your
password.',
```

```
    'weak-password' || 'password-does-not-meet-requirements' =>
      'Use at least 8 characters and one uppercase letter (A–Z).',
    'invalid-email' => 'Enter a valid email address.',
    'network-request-failed' => 'Check your internet connection
and try again.',
    'too-many-requests' => 'Too many attempts. Please try again
later.',
    'requires-recent-login' ||
    'user-token-expired' ||
    'invalid-user-token' => 'Please log out and log in again, then
retry.',
    'user-disabled' => 'This account has been disabled.',
    'operation-not-allowed' =>
      'Email sign-in is not available. Please try again later.',
    _ => 'Unable to complete this request. Please try again.',
  };
  @override
  void dispose() {
    _subscription.cancel();
    super.dispose();
  }
}
```

4.1 Виклик репозиторію та відображення даних

Наведений метод екрана входу спочатку перевіряє Form, потім викликає repository.signIn(). runAction() керує завантаженням, блокує повторну дію та показує AuthFailure. Після успішного входу AuthGate змінює екран відповідно до актуального currentUser.

Файл: lib/features/auth/login_screen.dart — метод _login()

```
Future<void> _login() async {
  if (busy || !_formKey.currentState!.validate()) return;
  final scope = AuthScope.of(context);
  if (await runAction(
    () =>
        scope.repository.signIn(email: _email.text, password:
_password.text),
  )) {
    await scope.analytics.login();
  }
}
```

SessionScreen слухає той самий репозиторій. Після зміни профілю ListenableBuilder отримує нову модель користувача й оновлює вміст без повторного створення маршруту та без втрати локального стану поливу.

Файл: lib/features/session/session_screen.dart — підписка на дані

```
@override
Widget build(BuildContext context) {
  final repository = AuthScope.of(context).repository;
  return ListenableBuilder(
    listenable: repository,
    builder: (context, _) {
      final user = repository.currentUser;
      if (user == null) return const SizedBox.shrink();
      return _buildSession(context, user);
    },
  );
}
```

Файл: lib/features/profile/profile_screen.dart — виведення імені та email

```
Text(
  user.name,
  textAlign: TextAlign.center,
  style: sproutText(24, weight: FontWeight.w700),
),
const SizedBox(height: 4),
Text(
  user.email,
  textAlign: TextAlign.center,
  style: sproutText(14, color: SproutColors.profileMuted),
),
```

На рис. 4 наведено профіль із збереженими даними та результат окремої перевірки оновлення імені: після встановлення Alex Green на Home відобразилося «Hi, Alex». Це різні стани тестування одного сценарію, а не фіксовані демонстраційні дані.

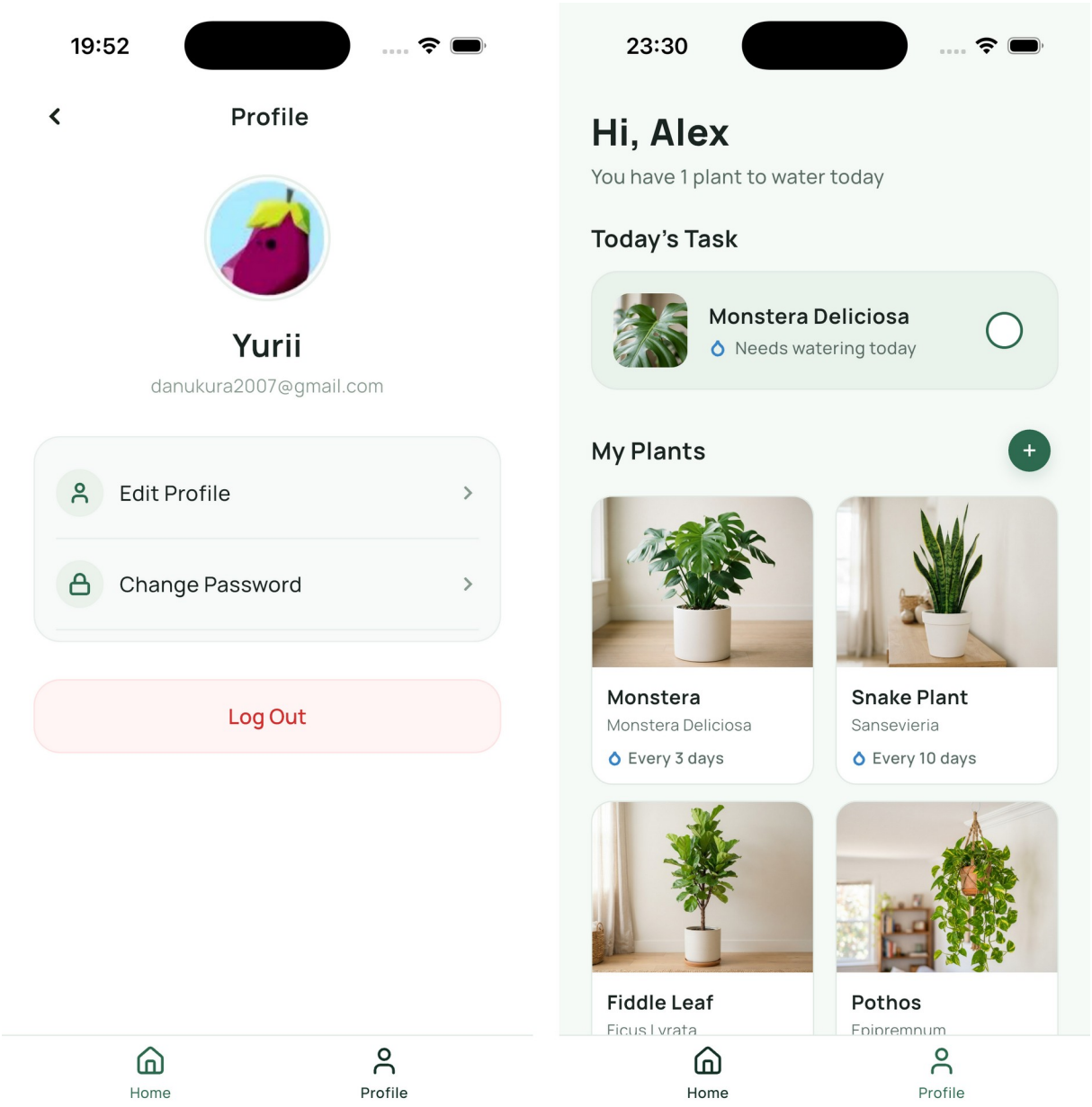


Рис. 4. Профіль користувача та Home після оновлення імені

4.2 Редагування профілю та вибір аватара

Edit Profile показує поточне ім’я, email лише для читання та фото. Натискання Change Photo відкриває нижню панель із вісьмома погодженими аватарами-персонажами. Вибір до Save Changes є чернеткою; повернення без збереження його скасовує. Під час запису заблоковано повторне надсилання та повернення, щоб незавершений запит не перезаписав наступні зміни.

У Firebase зберігається HTTPS-посилання на вибраний PNG у публічному репозиторії. У застосунок також додано локальні резервні зображення. Власні файли з галереї не завантажуються. Редактор та набір аватарів показано на рис. 5.

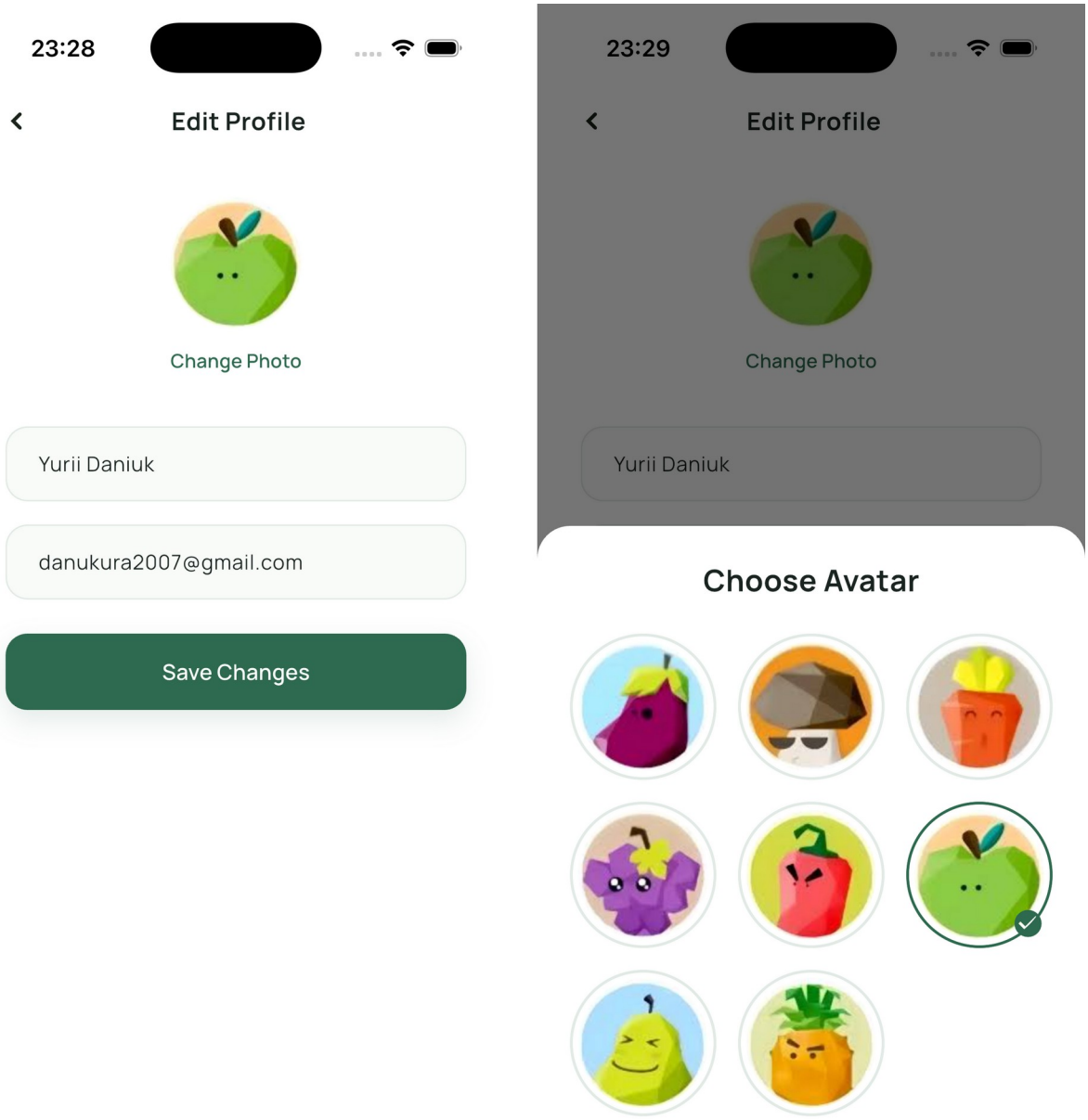


Рис. 5. Edit Profile та вибір одного з восьми HTTPS аватарів

4.3 Зміна та відновлення пароля

Change Password перевіряє Current Password, New Password і Confirm New Password. Після валідації репозиторій виконує reauthenticateWithCredential() та updatePassword(). Неправильний поточний пароль і потреба свіжого входу мають зрозумілі повідомлення; після успіху поля очищуються.

Forgot Password перевіряє email та викликає sendPasswordResetEmail(). Підтвердження не розкриває існування акаунта й очищується при зміні email або новій спробі. Лист відкриває стандартну сторінку Firebase. Екран запиту показано на рис. 6.

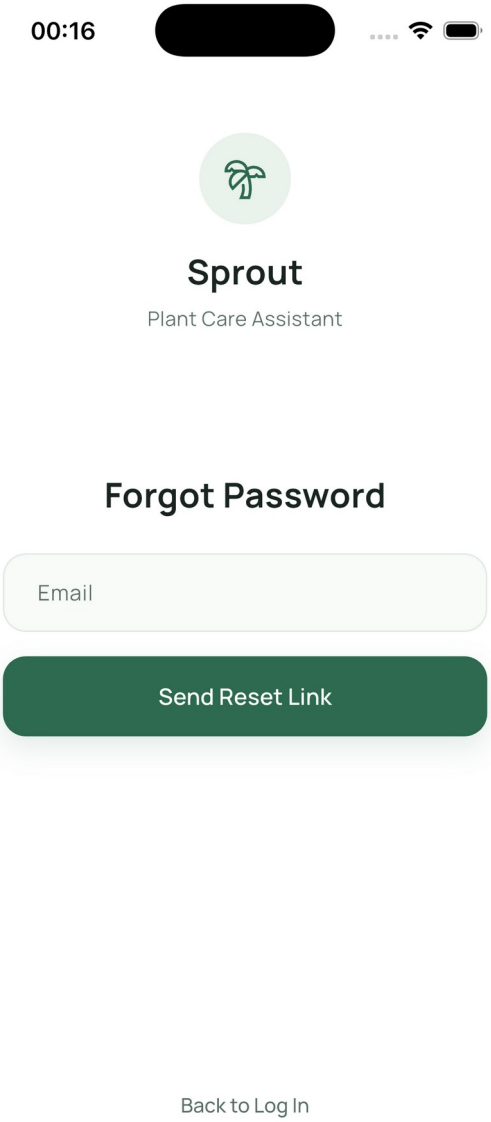


Рис. 6. Екран відновлення пароля через електронну пошту

5 Валідація форм реєстрації та входу

Форми входу, реєстрації, відновлення пароля, редагування імені й зміни пароля використовують Form. Власний SproutTextField побудовано на TextFormField і передає йому validator. GlobalKey зберігається у State. Порожнє ім’я, некоректний email або новий пароль, що не відповідає політиці, не надсилаються до Firebase.

Нижче наведено повний поточний код екрана реєстрації. Кнопка та відправлення з клавіатури використовують один метод _signup(). Поки busy дорівнює true, повторні натискання й повернення блокуються, а SproutButton показує індикатор завантаження.

Файл: lib/features/auth/signup_screen.dart

```
import 'package:flutter/cupertino.dart';

import '../../core/widgets/auth_links.dart';
import '../../core/widgets/auth_page.dart';
import '../../core/widgets/sprout_button.dart';
import '../../core/widgets/sprout_text_field.dart';
import 'auth_scope.dart';
import 'auth_action.dart';
import 'validation/auth_validators.dart';
import 'forgot_password_screen.dart';

class SignupScreen extends StatefulWidget {
  const SignupScreen({super.key});

  @override
  State<SignupScreen> createState() => _SignupScreenState();
}

class _SignupScreenState extends State<SignupScreen>
    with AuthAction<SignupScreen> {
  final _formKey = GlobalKey<FormState>();
  final _name = TextEditingController();
  @override
  String get analyticsScreen => 'signup';
  final _email = TextEditingController();
  final _password = TextEditingController();

  @override
  void dispose() {
    _name.dispose();
    _email.dispose();
    _password.dispose();
    super.dispose();
  }
```

```
  Future<void> _signup() async {
    if (busy || !_formKey.currentState!.validate()) return;
    final scope = AuthScope.of(context);
    if (await runAction(
      () => scope.repository.signUp(
        name: _name.text,
        email: _email.text,
        password: _password.text,
      ),
    )) {
      await scope.analytics.signup();
    }
  }

  @override
  Widget build(BuildContext context) => PopScope(
    canPop: !busy,
    child: AbsorbPointer(
      absorbing: busy,
      child: AuthPage(
        form: Form(
          key: _formKey,
          child: Column(
            children: [
              SproutTextField(
                label: 'Name',
                controller: _name,
                enabled: !busy,
                validator: AuthValidators.name,
              ),
              const SizedBox(height: 16),
              SproutTextField(
                label: 'Email',
                enabled: !busy,
                validator: AuthValidators.email,
                controller: _email,
                keyboardType: TextInputType.emailAddress,
              ),
              const SizedBox(height: 16),
              SproutTextField(
                label: 'Password',
                enabled: !busy,
                validator: AuthValidators.newPassword,
                controller: _password,
                password: true,
                textInputAction: TextInputAction.done,
                onSubmitted: (_) => _signup(),
              ),
              const SizedBox(height: 16),
              ActionError(actionError),
              SproutButton(label: 'Sign Up', loading: busy,
onPressed: _signup),
            ],
```

```
          ),
        ),
        footer: Column(
          children: [
            AuthLink(
              text: 'Forgot password?',
              onPressed: () => Navigator.of(context).push(
                CupertinoPageRoute<void>(
                  builder: (_) => const ForgotPasswordScreen(),
                ),
              ),
            ),
            AccountLink(
              prompt: 'Already have an account? ',
              action: 'Log In',
              onPressed: () => Navigator.of(context).pop(),
            ),
          ],
        ),
      ),
    ),
  );
}
```

Спільні правила перевірки винесено в AuthValidators. Для входу використовується requiredPassword(), а для реєстрації та нового пароля — newPassword(). Помилки серверної автентифікації обробляються окремо у репозиторії, оскільки правильний формат не доводить правильність облікових даних.

Файл: lib/features/auth/validation/auth_validators.dart

```
abstract final class AuthValidators {
  static String? name(String? value) =>
      value == null || value.trim().isEmpty ? 'Enter your name.' :
null;
  static String? email(String? value) {
    if (value == null || value.trim().isEmpty) return 'Enter your
email.';
    return RegExp(r'^[^\s@]+@[^\s@]+\.[^\s@]+
$').hasMatch(value.trim())
        ? null
        : 'Enter a valid email address.';
  }

  static String? requiredPassword(String? value) =>
      value == null || value.isEmpty ? 'Enter your password.' :
null;
  static String? newPassword(String? value) {
    if (value == null || value.length < 8) return 'Use at least 8
characters.';
    return RegExp(r'[A-Z]').hasMatch(value)
        ? null
```

```
            : 'Include at least one uppercase letter (A-Z).';
    }
}
```

На рис. 7 наведено два різні види помилок: локальна перевірка порожніх полів реєстрації та повідомлення про відсутність мережевого з’єднання під час входу. Після помилки користувач може виправити дані або повторити спробу.

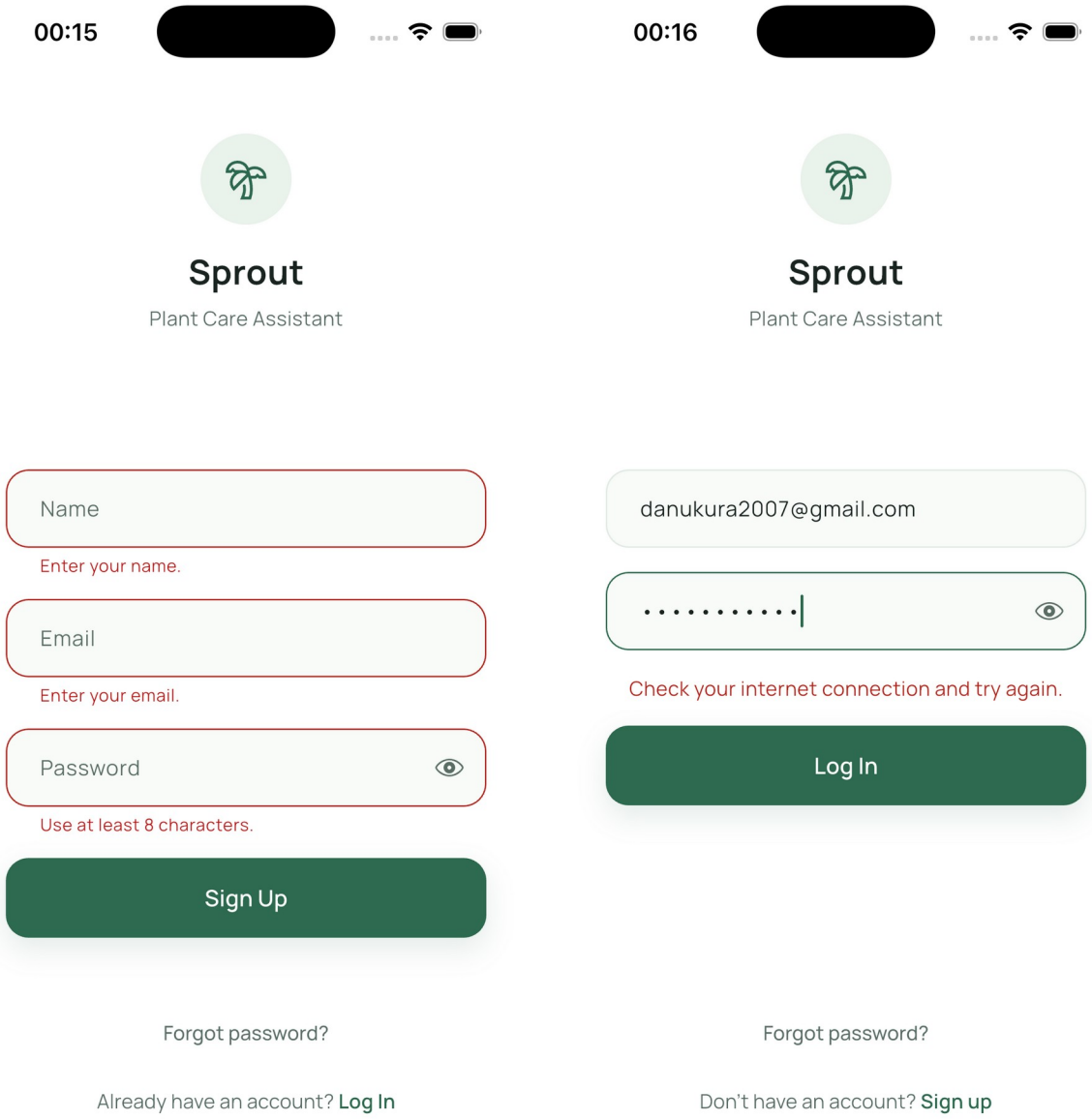


Рис. 7. Локальна валідація реєстрації та повідомлення про помилку мережі

6 Вміст файлу main.dart після змін

Точка входу запускає SproutBootstrap. Він один раз ініціалізує Firebase у initState(), підключає Crashlytics та створює репозиторій. FutureBuilder показує індикатор очікування або екран із Retry у разі помилки старту. У dispose() репозиторій звільняє підписку. Нижче наведено весь main.dart без скорочень.

Файл: lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_analytics/firebase_analytics.dart';

import 'app.dart';
import 'firebase_options.dart';
import 'core/services/analytics_service.dart';
import 'core/services/crash_reporting_service.dart';
import 'core/theme/sprout_theme.dart';
import 'features/auth/data/auth_repository.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const SproutBootstrap());
}

class SproutBootstrap extends StatefulWidget {
  const SproutBootstrap({super.key});
  @override
  State<SproutBootstrap> createState() => _SproutBootstrapState();
}

class _SproutBootstrapState extends State<SproutBootstrap> {
  late Future<FirebaseAuthRepository> _startup;
  FirebaseAuthRepository? _repository;
  @override
  void initState() {
    super.initState();
    _startup = _initialize();
  }

  Future<FirebaseAuthRepository> _initialize() async {
    await Firebase.initializeApp(
      options: DefaultFirebaseOptions.currentPlatform,
    );
    CrashReportingService.install();
    if (const bool.fromEnvironment('CRASHLYTICS_TEST')) {
      await CrashReportingService.sendTestError();
    }
    return _repository = FirebaseAuthRepository();
  }

  @override
```

```
  void dispose() {
    _repository?.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) =>
FutureBuilder<FirebaseAuthRepository>(
    future: _startup,
    builder: (context, snapshot) {
      final repository = snapshot.data;
      if (repository != null) {
        return SproutApp(
          auth: repository,
          analytics: AnalyticsService(FirebaseAnalytics.instance),
        );
      }
      return MaterialApp(
        theme: SproutTheme.light,
        debugShowCheckedModeBanner: false,
        home: Scaffold(
          body: SafeArea(
            child: Center(
              child: Padding(
                padding: const EdgeInsets.all(32),
                child: snapshot.hasError
                    ? Column(
                        mainAxisSize: MainAxisSize.min,
                        children: [
                          const Text(
                            'Unable to start Sprout. Please try
again.',
                            textAlign: TextAlign.center,
                          ),
                          const SizedBox(height: 16),
                          TextButton(
                            onPressed: () =>
                                setState(() => _startup =
_initialize()),
                            child: const Text('Retry'),
                          ),
                        ],
                      )
                    : const CircularProgressIndicator(),
              ),
            ),
          ),
        ),
      );
    },
  );
}
```

7 Структура проєкту після змін

Основний код розділено за функціями: auth, home, profile і session. Каталог core містить спільні сервіси, тему та віджети. Повну структуру Dart-коду та основні конфігураційні файли наведено нижче; згенеровані каталоги збірки й кеші не показано.

Файл: структура робочого Flutter-проєкту

```
sprout/
├── lib/
│   ├── core/
│   │   ├── services/
│   │   │   ├── analytics_service.dart
│   │   │   ├── crash_reporting_service.dart
│   │   │   └── screen_tracking_scope.dart
│   │   ├── theme/
│   │   │   └── sprout_theme.dart
│   │   └── widgets/
│   │       ├── auth_links.dart
│   │       ├── auth_page.dart
│   │       ├── brand_header.dart
│   │       ├── demo_message.dart
│   │       ├── page_header.dart
│   │       ├── sprout_button.dart
│   │       ├── sprout_icon.dart
│   │       ├── sprout_text_field.dart
│   │       └── user_avatar.dart
│   ├── features/
│   │   ├── auth/
│   │   │   ├── data/
│   │   │   │   └── auth_repository.dart
│   │   │   ├── validation/
│   │   │   │   └── auth_validators.dart
│   │   │   ├── auth_action.dart
│   │   │   ├── auth_gate.dart
│   │   │   ├── auth_scope.dart
│   │   │   ├── forgot_password_screen.dart
│   │   │   ├── login_screen.dart
│   │   │   └── signup_screen.dart
│   │   ├── home/
│   │   │   └── home_screen.dart
│   │   ├── profile/
│   │   │   ├── data/
│   │   │   │   └── avatar_catalog.dart
│   │   │   ├── change_password_screen.dart
│   │   │   ├── edit_profile_screen.dart
│   │   │   └── profile_screen.dart
│   │   └── session/
│   │       ├── app_user.dart
│   │       └── session_screen.dart
```

```
│   ├── app.dart
│   ├── firebase_options.dart
│   └── main.dart
├── test/
│   ├── support/
│   │   └── fake_auth_repository.dart
│   ├── auth_repository_test.dart
│   ├── review_fixes_test.dart
│   └── widget_test.dart
├── assets/
│   ├── avatars/   (8 PNG та опис походження)
│   ├── fonts/     (Manrope)
│   ├── icons/     (локальні SVG)
│   └── images/    (фотографії рослин)
├── ios/
│   ├── Runner/GoogleService-Info.plist
│   ├── Runner.xcodeproj/
│   └── Runner.xcworkspace/
├── firebase.json
├── pubspec.yaml
├── pubspec.lock
├── analysis_options.yaml
├── LAB3_PLAN.md
└── LAB4_PLAN.md
```

firebase_options.dart і GoogleService-Info.plist пов’язують клієнт із Firebase. auth_repository.dart містить операції акаунта; auth_gate.dart визначає доступний стек екранів; auth_action.dart керує станами запитів і відстеженням маршрутів. app_user.dart описує дані профілю, а session_screen.dart оновлює Home/Profile за змінами репозиторію.

У core/services розміщені AnalyticsService, CrashReportingService та ScreenTrackingScope. У core/widgets — спільні кнопки, поля, заголовки, аватар та компонування форм. Тести відокремлено від робочого коду; FakeAuthRepository дає змогу перевіряти сценарії без реального Firebase.

Публічний репозиторій проєкту: https://github.com/yuriiDaniuk/sprout. У ньому опубліковано базовий проєкт і погоджені PNG аватарів; поточна реалізація лабораторної № 4 та цей звіт підготовлені локально.

8 Перевірка результатів

20 жовтня 2026 року повторно виконано flutter analyze та flutter test. Аналіз завершився повідомленням «No issues found!», усі 30 тестів пройшли успішно. Результати автоматизованих перевірок наведено в табл. 1.

Таблиця 1 — Результати автоматизованої перевірки

Перевірка	Що перевірено	Результат
flutter analyze	Статичний аналіз Dart-коду	Без зауважень
flutter test	3 тестові файли з 30 перевірками	30 успішних
Форми й запити	Валідація, видимість пароля, завантаження та захист від повторного надсилання	Успішно
Сеанс і профіль	Навігація, оновлення імені, чернетка аватара, повторна спроба після помилки	Успішно
Граничні сценарії	Часткова реєстрація, незавершений профіль, блокування повернення, вузький екран і шрифт 200%	Успішно
Analytics	Повернення на маршрути та вкладки без дублів від перебудови	Успішно

Ручні перевірки виконувалися на iPhone 17 Pro з iOS 26.5. Підтверджено реєстрацію, вхід і вихід, відновлення сеансу, оновлення імені та аватара, скасування чернетки й актуальне привітання Home. Успішні зміну пароля та відновлення через лист підтверджено користувачем.

Перевірено валідацію, повідомлення про помилки облікових даних і мережі, блокування дубльованих запитів та повторні спроби. Тести також охоплюють клавіатуру, прокручування й збільшений текст. Усі скриншоти застосунку в цьому звіті отримано лише з iPhone 17 Pro.

Акаунт, профіль і телеметрія працюють із Firebase. Рослини та полив залишаються локальними. Cloud Firestore, push-сповіщення й завантаження власних фото відкладені відповідно до винятків методички.

ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що означають складові номера версії MAJOR.MINOR.PATCH?

MAJOR збільшують для несумісних змін API, MINOR — для додавання сумісної функціональності, PATCH — для сумісних виправлень помилок. Наприклад, у версії 1.2.3 ці складові дорівнюють 1, 2 і 3. Дотримання правил залежить від автора пакета, тому після оновлення потрібні перевірки.

2. Які діапазони версій задають записи ^1.2.0 та ^0.2.3?

^1.2.0 означає >=1.2.0 <2.0.0, а ^0.2.3 — >=0.2.3 <0.3.0. Для початкової мажорної версії 0 сумісна верхня межа вужча. Остаточний вибір також обмежується вимогами SDK та інших залежностей.

3. Як у pubspec.yaml зафіксувати точну версію пакета без автооновлень?

Потрібно вказати номер без символу ^ і без діапазону, наприклад firebase_core: 5.0.0. Конкретні розв’язані версії усіх залежностей зберігаються у pubspec.lock. Пакети не оновлюються самі під час запуску застосунку.

4. Чим відрізняються команди flutter pub get та flutter pub upgrade?

pub get отримує залежності та за можливості зберігає сумісні версії з pubspec.lock. pub upgrade намагається вибрати найновіші версії, які дозволені поточними обмеженнями проєкту, і оновлює lock-файл. Обидві команди враховують взаємні обмеження залежностей.

5. Який офіційний репозиторій є основним джерелом пакетів для Flutter?

Основним репозиторієм є pub.dev. На сторінках пакетів наведено опис, версії, підтримувані платформи, документацію й приклади. Перед підключенням слід перевірити сумісність пакета з проєктом.

6. Як підключити локальний пакет із сусідньої директорії?

У dependencies для потрібного пакета задають поле path, наприклад my_package: з вкладеним path: ../my_package. Шлях відраховується від pubspec.yaml застосунку. Після зміни конфігурації виконують flutter pub get.

7. Як додати залежність із Git?

У dependencies для пакета задають git, усередині — url репозиторію та, за потреби, ref гілки, тегу або коміту. Якщо пакет лежить у підкаталозі, вказують також path. Фіксація коміту дає змогу відтворити саме потрібну редакцію коду.

8. Для чого використовують платформні канали та як працює взаємодія через MethodChannel?

Канали дають Dart доступ до нативних API або SDK. Dart і нативний код створюють канал з однаковою назвою; invokeMethod() надсилає назву операції та аргументи, обробник виконує її й повертає результат або помилку. Обмін асинхронний, а значення мають підтримуватися кодеком.

9. Чим відрізняються помилки PlatformException та MissingPluginException?

PlatformException означає, що виклик отримав повідомлення про помилку від платформного обробника. MissingPluginException означає відсутність зареєстрованої реалізації для каналу чи методу. У другому випадку перевіряють підтримку платформи, реєстрацію плагіна та повну перебудову застосунку.

10. Чому перед підключенням Firebase-плагіна потрібно перевіряти підтримку цільової платформи? Які є альтернативи за відсутності підтримки?

Не кожний Firebase-плагін має реалізацію для всіх платформ, тому наявність Dart API ще не гарантує роботу на цільовому пристрої. За відсутності підтримки можна використати інший сервіс, зокрема Amplitude для аналітики чи Sentry для помилок, або доступний REST API. Альтернативи також потрібно перевіряти на сумісність.

11. Яку роль відіграють firebase_core, firebase_options.dart і виклик Firebase.initializeApp()?

firebase_core надає базовий API ініціалізації Firebase. firebase_options.dart містить згенеровані параметри для зареєстрованих платформ. Firebase.initializeApp() створює налаштований екземпляр Firebase; перед роботою інших сервісів потрібно дочекатися завершення цього Future.

12. Що таке подія та її параметри у Firebase Analytics? Чим автоматично зібрані події відрізняються від подій, які реєструє розробник?

Подія описує дію або стан, а параметри уточнюють її контекст. Автоматичні події SDK створює сам за передбачених умов, наприклад для першого запуску чи активності сеансу. Специфічні дії Sprout і переходи між його Flutter-екранами реєструються з коду.

13. Як зареєструвати власну подію та організувати відстеження переходів між Flutter-екранами?

Власну подію передають через FirebaseAnalytics.instance.logEvent(name: ..., parameters: ...). Екрани можна відстежувати через logScreenView() або налаштований observer навігації. У Sprout RouteAware відстежує відкриття й повернення на маршрут, а зміна вкладок окремо викликає AnalyticsService.screen().

14. Яку інформацію містять звіти Firebase Crashlytics і як вони допомагають знаходити причини помилок?

Звіт містить опис винятку, стек викликів, версію застосунку та характеристики середовища. Схожі події об’єднуються в issue з даними про повторюваність. Читабельний стек допомагає знайти файл і рядок, із якого виникла помилка, як у контрольному звіті Sprout.

15. Чим відрізняються фатальні й нефатальні помилки?

Фатальна помилка призводить до аварійного завершення або позначається як критична у звіті Crashlytics. Нефатальна помилка реєструє проблему, після якої застосунок може продовжити роботу. Наприклад, невдалий запит можна обробити повідомленням і дозволити повторну спробу.

16. Чи завершує застосунок виклик recordError() із fatal: true?

Ні. Параметр fatal: true позначає звіт як фатальний, але сам recordError() не припиняє виконання процесу. Реальне аварійне завершення та класифікація запису в системі звітності — різні дії.

17. Чим відрізняються методи log() та recordError() у Crashlytics?

log() додає діагностичний запис у журнал, який може супроводжувати звіт. recordError() реєструє саму помилку зі стеком та додатковою причиною. Виклик лише log() не створює окремого звіту про виняток.

18. Для чого налаштовують FlutterError.onError та PlatformDispatcher.instance.onError?

FlutterError.onError отримує помилки, які перехоплює Flutter під час виконання своїх callback, компонування чи малювання. PlatformDispatcher.instance.onError обробляє необроблені помилки кореневого ізоляту поза цим механізмом. Обидва обробники потрібні для передавання відповідних категорій помилок до Crashlytics.

19. Які способи автентифікації підтримує Firebase Authentication?

Серед способів — email/password, вхід за номером телефону, анонімний вхід, зовнішні провайдери, зокрема Google, Apple, Facebook і GitHub, а також custom token. Набір доступних способів залежить від платформи й конфігурації. У Sprout увімкнено лише погоджений Email/Password.

20. Чим вхід за email і паролем відрізняється від входу за номером телефону?

Email/password перевіряє постійні облікові дані акаунта. Телефонний вхід підтверджує доступ до номера через SMS-код і відповідний credential; звичайна операція зміни пароля до нього не застосовується. Це різні провайдери, які за потреби можна зв’язати з одним користувачем.

21. Яке призначення Firebase ID token, refresh token, токенів зовнішніх провайдерів і Firebase custom token?

ID token — підписаний Firebase токен, що підтверджує особу для сервісів або власного бекенду. Refresh token дозволяє отримувати нові ID-токени без повторного введення пароля. Токени зовнішнього провайдера використовують для отримання Firebase credential. Custom token створює довірений сервер для входу у Firebase, після якого клієнт отримує звичайні токени сеансу.

22. Для чого використовують зв’язування кількох способів входу з одним обліковим записом?

Зв’язування дає користувачу змогу входити різними провайдерами зі збереженням одного uid і спільних даних. Для цього нові облікові дані приєднують до поточного користувача, наприклад через linkWithCredential(). Окрема реєстрація кожним способом без зв’язування може створити різні акаунти.

23. Які дані профілю зберігає Firebase Authentication?

Основні поля — uid, email, displayName, photoURL, номер телефону та дані провайдерів залежно від способу входу й заповнення. Сервіс також має стан підтвердження email та інші атрибути акаунта. Він не є сховищем довільних даних рослин, статистики чи налаштувань застосунку.

24. Як отримати поточного користувача та змінити його ім’я або URL фотографії?

Поточного користувача отримують через FirebaseAuth.instance.currentUser, значення може бути null. Ім’я змінюють методом updateDisplayName(), а посилання на фото — updatePhotoURL(). Після завершення слід оновити дані інтерфейсу через відстеження стану або актуальну модель; зображення при цьому не завантажується у Storage.

25. Як виконується зміна пароля та чим вона відрізняється від відновлення забутого пароля?

Для зміни пароля активного користувача викликають updatePassword(), за потреби після повторної автентифікації. Для забутого пароля sendPasswordResetEmail() надсилає лист із посиланням для встановлення нового значення. У Sprout зміна підтверджується поточним паролем, а відновлення виконується на стандартній сторінці Firebase.

26. Що означає помилка requires-recent-login і як її опрацювати?

Чутлива операція потребує недавнього підтвердження особи, а поточний сеанс такого підтвердження вже не має. Потрібно отримати credential користувача, виконати reauthenticateWithCredential() і повторити операцію або запропонувати повторний вхід. Повторення того самого запиту без підтвердження проблему не усуне.

27. Для чого використовують Form і GlobalKey<FormState>?

Form об’єднує поля, їхню перевірку та стан. GlobalKey<FormState> дає доступ до відповідного FormState, зокрема до validate(), save() і reset(). У Sprout він використовується для перевірки всіх полів перед серверним запитом.

28. Чому ключ GlobalKey<FormState> не слід створювати заново в методі build()?

Новий ключ змінює ідентичність форми під час кожної побудови й може спричинити втрату її стану та повідомлень. Ключ створюють один раз як поле State, а build() використовує вже наявний екземпляр. Це особливо важливо під час зміни busy або показу помилки.

29. Як працюють TextFormField.validator та FormState.validate()?

validator отримує значення поля та повертає null, якщо воно коректне, або текст помилки. FormState.validate() викликає перевірки всіх підключених полів, оновлює їхнє відображення та повертає true лише за відсутності помилок. Після true можна починати асинхронний запит.

30. Чим локальна валідація введених даних відрізняється від перевірок Firebase Authentication та оброблення помилок запиту?

Локальна валідація перевіряє обов’язковість, формат email, політику й підтвердження пароля до звернення до мережі. Firebase перевіряє правильність облікових даних, стан акаунта та серверні обмеження. Навіть валідна форма може отримати помилку мережі або автентифікації, яку потрібно окремо пояснити користувачу.

31. Навіщо виносити взаємодію з Firebase в окремий клас AuthRepository?

Репозиторій відокремлює представлення від SDK та бізнес-логіки, централізує оброблення помилок і модель користувача. Екрани викликають зрозумілі операції signIn(), signUp() та updateProfile(). У тестах реалізацію можна замінити підставною, щоб перевіряти UI без мережі та реальних паролів.

32. Як інтерфейс повинен відображати виконання асинхронного запиту, його успішне завершення та помилку?

Під час запиту показують завантаження, блокують повторне надсилання та небезпечне закриття форми. Після успіху оновлюють дані, переходять на потрібний екран або показують підтвердження. Після помилки відновлюють доступність дій, зберігають введені дані та показують зрозуміле повідомлення з можливістю повторити спробу; перед оновленням State після await перевіряють mounted.

ВИСНОВКИ

Під час виконання лабораторної роботи я підключив Sprout до Firebase та замінив демонстраційну автентифікацію справжньою роботою з акаунтами. Реалізував реєстрацію й вхід за email та паролем, відновлення доступу листом, вихід і відновлення сеансу після перезапуску.

Я додав редагування імені, вибір одного з восьми HTTPS-аватарів та зміну пароля з повторною перевіркою поточних облікових даних. Зміни профілю відображаються на Home та Profile. У формах реалізував валідацію, стан завантаження, блокування повторного надсилання і зрозумілі повідомлення про помилки.

Взаємодію з Firebase я виніс у AuthRepository, а сервіси аналітики та діагностики — у спільні компоненти. Firebase Analytics отримав події використання, а Crashlytics — контрольну нефатальну помилку з читабельним стеком. Статичний аналіз завершився без зауважень, усі 30 автоматизованих тестів пройшли успішно; основні сценарії перевірено на iPhone 17 Pro.

У результаті я закріпив навички інтеграції Flutter із Firebase, організації асинхронних операцій, перевірки форм і розділення інтерфейсу та бізнес-логіки. Дані рослин і поливу залишив локальними: хмарна база, push-сповіщення та завантаження власних зображень належать до наступних етапів.